Lecture 8 — Recursion & Backtracking

> Year Curriculum · Prerequisites: Weeks 2–3 (C++/STL, complexity & complete search) · Estimated time: 2 × 90 min

1. Overview

In Week 3 we met *complete search*: when the input is small, try every candidate and keep the best. For simple problems a few nested loops do the job, but loops break down as soon as the *number* of nested choices depends on the input ("choose a subset of n items", "order n items", "place n queens"). Recursion handles a variable number of nested choices naturally: each recursive call makes **one** decision and delegates the rest.

Backtracking is recursion plus a discipline: build a candidate solution incrementally, and as soon as a partial candidate cannot possibly be completed into a valid answer, abandon it ("backtrack") and try something else. With good pruning, backtracking explores a tiny fraction of the raw search space; that is how we can count all 8-queens placements with about two thousand recursive calls instead of billions.

After this lecture you should be able to:

- write correct recursive functions (base case, progress, state restoration);
- generate all subsets, permutations and combinations of a set, with and without duplicates;
- apply the *choose → explore → un-choose* template to constraint problems such as N-Queens;
- design pruning rules and estimate running time by counting nodes in the search tree.

This lecture is the foundation for Week 9 (bitmask enumeration, the iterative alternative to subset recursion), Week 14 (memoization = recursion + caching) and Week 26 (bitmask DP, which replaces exponential search with DP over subsets).

2. Recursion fundamentals

A recursive function solves an instance of a problem by solving smaller instances of the same problem. Two ingredients are mandatory:

1. **Base case(s)**: instances small enough to answer directly.
2. **Recursive case**: express the answer in terms of strictly smaller instances, so that every chain of calls eventually reaches a base case.

```cpp
long long fact(int n) {
    if (n == 0) return 1;      // base case
    return n * fact(n - 1);    // recursive case, n-1 < n
}
```

2.1 The call stack

Each active call has its own *stack frame* containing its parameters, local variables and return address. Frames are pushed on call and popped on return, so the memory used at any moment is proportional to the current **depth** of recursion, not to the total number of calls. Most judges (Codeforces, AtCoder, CSES) give you a large stack, and depths around 10^5–10^6 with small frames are usually fine. Some judges (and default Windows builds) have a 1 MB stack, where depth $\sim 10^5$ with a few locals per frame can already crash with a runtime error. If you need recursion depth of 10^6 or more, prefer an iterative version.

2.2 Recursion trees

The best mental model for analysing recursive code is the **recursion tree**: one node per call, with children for the calls it makes. Then:

- **total time** ≈ (number of nodes) × (non-recursive work per node);
- **memory** ≈ (maximum depth) × (frame size) + any global structures.

For naive Fibonacci, `fib(n)` calls `fib(n-1)` and `fib(n-2)`, so the tree has about $\varphi^n \approx 1.618^n$ nodes — exponential, because the same subproblems appear over and over. In Week 14 we fix that with memoization. In this lecture the situation is different: the leaves of our trees are *distinct* candidates that we genuinely want to examine (or prune).

3. Brute force as a decision tree

Almost every complete search can be phrased as a sequence of decisions:

Problem	Decision at depth *i*	Branching	Leaves
Subsets of n items	include item *i*?	2	2^n
Permutations of n items	which unused item goes in position *i*	n − i	n!
Strings of length L over alphabet Σ	letter at position *i*	$\lvert\Sigma\rvert$	$\lvert\Sigma\rvert^L$
N-Queens	column of queen in row *i*	≤ n	$\leq n^n$

The recursion depth equals the number of decisions; the *state* is the list of decisions made so far (plus any derived bookkeeping, such as "which columns are already used").

3.1 Generating all subsets

```
int n;
vector<int> cur;

void subsets(int i) {          // decide about element i (1-indexed)
    if (i == n + 1) {          // all decisions made: cur is a subset
        process(cur);
        return;
    }
    subsets(i + 1);            // branch 1: exclude i
    cur.push_back(i);          // branch 2: include i
    subsets(i + 1);
    cur.pop_back();            // restore state for the caller
}
```

For n = 3 the output order is `{} {3} {2} {2 3} {1} {1 3} {1 2} {1 2 3}`. The tree is a complete binary tree with 2^n leaves and $2^{n+1} - 1$ nodes. If processing a subset costs O(n), the total is **$O(n \cdot 2^n)$**. With 10^8 simple operations per second, this is fine up to roughly n = 20–23.

3.2 Generating all permutations

```
vector<int> cur;
vector<bool> used;   // used[v] == true if v is already in cur

void perms() {
    if ((int)cur.size() == n) { process(cur); return; }
    for (int v = 1; v <= n; v++) {
        if (used[v]) continue;
        used[v] = true;  cur.push_back(v);    // choose
        perms();                              // explore
        used[v] = false; cur.pop_back();      // un-choose
    }
}
```

The number of nodes at depth k is n!/(n−k)!, so the total number of nodes is $n! \cdot \Sigma\, 1/j! < e \cdot n!$. Each node does an O(n) loop, giving **$O(n \cdot n!)$** in the worst case. $10! \approx 3.6 \cdot 10^6$ is comfortable; $12! \approx 4.8 \cdot 10^8$ is already too slow for most time limits unless pruning removes almost everything.

The STL alternative is `next_permutation`, which transforms a sequence into the lexicographically next one and returns `false` after the last. Start from a **sorted** sequence, otherwise you miss permutations. It also handles duplicates correctly (distinct permutations only), which makes it ideal for CSES 1622 *Creating Strings*. Its weakness is that it cannot skip a whole family of permutations sharing a bad prefix, which a recursive generator can do with one `continue`.

3.3 Combinations and avoiding duplicates

To generate k-element subsets, or multisets where order does not matter, pass a `start` index so that elements are always chosen in increasing index order. This produces each combination exactly once:

```
void comb(int start, int k) {
    if (k == 0) { process(cur); return; }
    for (int i = start; i < n; i++) {
        cur.push_back(a[i]);
        comb(i + 1, k - 1);
        cur.pop_back();
    }
}
```

When the input contains repeated values (say `{1, 2, 2}`) and you want *distinct* outputs, sort the input and, at each level of the tree, skip a value equal to the one just tried at the same level: `if (i > start && a[i] == a[i-1]) continue;`. Both `{1,2}` subsets would otherwise appear twice.

4. The backtracking template

```
solve(state):
    if state is a complete solution: record it; return
    for each choice c that is valid in state:
        apply c to state         # choose
        solve(state)             # explore
        undo c                   # un-choose
```

Two design decisions dominate:

1. **What is the state, and how is it shared?** Keeping the state in global arrays (or passing it by reference) and undoing changes on return costs O(1) per change. Passing a `vector` *by value* to every call is simpler to get right but costs O(n) per node, which can multiply your running time by n.
2. **How do you check validity?** Maintain auxiliary arrays (used columns, used digits, current sum) so that "is this choice valid?" is O(1) rather than a scan of the partial solution.

4.1 Worked example: N-Queens with reserved squares (CSES 1624)

We must place 8 queens on an 8 × 8 board so that none attack each other, and some squares are reserved. Since two queens in the same row attack each other, there is exactly one queen per row; the decision at depth r is the column of the row-r queen.

Square (r, c) conflicts with an earlier queen if that queen shares the column, the "↘" diagonal (constant r − c) or the "↙" diagonal (constant r + c). For an n × n board, r + c ranges over 0 … 2n − 2 and r − c over −(n−1) … n−1, which we shift by n − 1 to get a valid index. Three boolean arrays give an O(1) check.

Trace of the first few steps on an empty 4 × 4 board:

```
row 0: try c=0  → Q...            ok
  row 1: c=0 col used, c=1 diag used, c=2 → Q... / ..Q.   ok
    row 2: c=0 col, c=1 diag(r+c=3), c=2 col, c=3 diag(r-c=-1) → dead end, backtrack
  row 1: c=3 → Q... / ...Q        ok
    row 2: c=1 → .Q..             ok
      row 3: every column attacked → backtrack
    ...
```

Every dead end prunes the entire subtree below it. On an empty 8 × 8 board the full search makes only 2,057 calls and finds 92 solutions, compared with $8^8 \approx 1.7 \cdot 10^7$ for "one queen per row, check at the end" and $C(64, 8) \approx 4.4 \cdot 10^9$ for "any 8 squares".

4.2 Pruning strategies

- **Feasibility pruning**: stop as soon as the partial state violates a constraint (the queens check above). The earlier you check, the bigger the subtree you remove.
- **Bound pruning (branch and bound)**: in an optimisation problem, if the cost so far (plus an optimistic estimate of the rest) is already no better than the best complete answer found, return.
- **Ordering**: sort the choices so that once one fails, all later ones fail too, and `break` instead of `continue`. In the combination-sum example below, candidates are sorted, so `a[i] > remaining` ends the loop.
- **Symmetry breaking**: if solutions come in symmetric families, generate one representative. For N-Queens on an empty board with even n, put the first queen in the left half and double the count.
- **Structural dead ends**: CSES 1625 *Grid Path Description* is the classic example — if the path reaches a cell where it cannot go forward but can go both left and right, the unvisited cells have been split into two regions and can never all be visited, so stop immediately. Pruning like this turns a hopeless search into one that runs in under a second.

4.3 Worked example: combination sum

Given distinct positive candidates and a target, list every multiset (repetition allowed) summing to the target. With candidates `{2, 3, 6, 7}` and target 7 the answers are `2 2 3` and `7`.

```
void go(int start, int remaining) {
    if (remaining == 0) { print(cur); return; }
    for (int i = start; i < (int)a.size(); i++) {
        if (a[i] > remaining) break;     // sorted → every later candidate fails too
        cur.push_back(a[i]);
        go(i, remaining - a[i]);         // i (not i+1): a[i] may be reused
        cur.pop_back();
    }
}
```

Passing `start` keeps the chosen values non-decreasing, so `{2,2,3}` is generated once rather than as three orderings. On this input the search visits only 10 nodes.

5. Divide and conquer recursion: Tower of Hanoi

Not every recursion is a search. Tower of Hanoi (CSES 2165) asks for the shortest sequence of moves that transfers n disks from peg 1 to peg 3. To move the largest disk we must first move the $n - 1$ smaller ones out of the way, then move the largest disk, then move the $n - 1$ disks back on top:

```
void hanoi(int n, int from, int to, int via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);
    moves.push_back({from, to});
    hanoi(n - 1, via, to, from);
}
```

The move count satisfies $T(n) = 2T(n-1) + 1$, $T(0) = 0$, so $T(n) = 2^n - 1$, which is also a lower bound, so the recursion is optimal.

6. Estimating running time

Before coding a search, bound the size of the tree:

n	2^n	n!
10	1,024	3,628,800
15	32,768	$\approx 1.3 \cdot 10^{12}$
20	$\approx 10^6$	$\approx 2.4 \cdot 10^{18}$
25	$\approx 3.4 \cdot 10^7$	—

Rules of thumb: **$2^n \cdot n$ is fine for $n \le 20$, $n! \cdot n$ for $n \le 10$.** Pruning often helps by orders of magnitude, but unless you can argue why the pruned tree is small, assume the worst case. When n is about 40 and you need subsets, the trick is **meet in the middle** (CSES 1628):

enumerate the 2^{20} subset sums of each half separately, sort one list, and combine with binary search or two pointers, for O(2^(n/2) · n) total.

7. Reference implementations

7.1 Subsets and permutations generator

Input `n`; prints all subsets of {1..n}, then all permutations.

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> cur;
vector<bool> used;

// Print all subsets of {1..n} (include/exclude decision tree)
void subsets(int i) {
    if (i == n + 1) {
        cout << "{";
        for (size_t k = 0; k < cur.size(); k++) cout << (k ? " " : "") << cur[k];
        cout << "}\n";
        return;
    }
    subsets(i + 1);          // exclude i
    cur.push_back(i);        // choose
    subsets(i + 1);          // include i
    cur.pop_back();          // un-choose
}

// Print all permutations of {1..n}
void perms() {
    if ((int)cur.size() == n) {
        for (int k = 0; k < n; k++) cout << cur[k] << " \n"[k == n - 1];
        return;
    }
    for (int v = 1; v <= n; v++) {
        if (used[v]) continue;
        used[v] = true; cur.push_back(v);
        perms();
        used[v] = false; cur.pop_back();
    }
}

int main() {
    cin >> n;
    used.assign(n + 1, false);
    subsets(1);
    perms();
}
```

7.2 CSES 1624 — Chessboard and Queens

```cpp
#include <bits/stdc++.h>
using namespace std;
// CSES 1624 - Chessboard and Queens: 8x8 board, '*' = reserved square.
string g[8];
bool colUsed[8], d1[15], d2[15];   // d1: r+c, d2: r-c+7
long long ways = 0;

void place(int r) {
    if (r == 8) { ways++; return; }
    for (int c = 0; c < 8; c++) {
        if (g[r][c] == '*' || colUsed[c] || d1[r + c] || d2[r - c + 7]) continue;
        colUsed[c] = d1[r + c] = d2[r - c + 7] = true;
        place(r + 1);
        colUsed[c] = d1[r + c] = d2[r - c + 7] = false;
    }
}

int main() {
    for (int i = 0; i < 8; i++) cin >> g[i];
    place(0);
    cout << ways << "\n";
}
```

On the CSES sample this prints 65; on an empty board, 92.

7.3 Combination sum with pruning

Input: `n target` followed by n distinct positive integers.

```cpp
#include <bits/stdc++.h>
using namespace std;
// Print every multiset of the (distinct, positive) candidates that sums to target.
// Each candidate may be reused. Sorted candidates let us prune with `break`.
vector<int> a, cur;
int target;
long long nodes = 0;

void go(int start, int remaining) {
    nodes++;
    if (remaining == 0) {
        for (size_t k = 0; k < cur.size(); k++) cout << cur[k] << " \n"[k + 1 ==
cur.size()];
        return;
    }
    for (int i = start; i < (int)a.size(); i++) {
        if (a[i] > remaining) break;          // pruning: all later values are larger too
        cur.push_back(a[i]);
        go(i, remaining - a[i]);              // i, not i+1: reuse allowed
        cur.pop_back();
```

```
    }
}

int main() {
    int n; cin >> n >> target;
    a.resize(n);
    for (auto &x : a) cin >> x;
    sort(a.begin(), a.end());
    go(0, target);
    cerr << "nodes visited: " << nodes << "\n";
}
```

8. Common mistakes

- **Forgetting to undo.** Every modification made before the recursive call (push, mark used, add to sum) must be reversed after it, on *every* path — including when you `return` early from inside the loop after finding an answer.
- **Missing or wrong base case.** Infinite recursion shows up as "Runtime Error" or "Memory Limit Exceeded" (the stack is memory). Check the off-by-one: does the base case fire at `i == n` or `i == n + 1` given your indexing?
- **Copying state on every call.** `void go(vector<int> cur)` copies the vector at each node. Pass by reference and undo, or make it global.
- **Checking constraints only at the leaves.** Correct but slow: N-Queens checked only at the end explores 8^8 leaves instead of ~2,000 nodes.
- **Duplicate outputs.** Generating combinations without a `start` index yields every ordering; repeated input values need sorting plus the "skip equal sibling" rule.
- **`next_permutation` on unsorted input** silently skips every permutation lexicographically smaller than the start.
- **Exceeding the stack** with depth ~10^6 or more, or with large local arrays inside a recursive function. Make big arrays global.
- **Underestimating the tree.** n = 12 permutations is about $5 \cdot 10^8$ leaves; always do the arithmetic first.

9. Practice problems

Warm-up:

1. **CSES 1622 — Creating Strings** — https://cses.fi/problemset/task/1622 (distinct permutations of a string; recursion or `next_permutation`).
2. **CSES 1623 — Apple Division** — https://cses.fi/problemset/task/1623 (2^n subsets, $n \le 20$).
3. **AtCoder ABC 114 C — 755** — https://atcoder.jp/contests/abc114/tasks/abc114_c (recursively generate numbers made of digits 3, 5, 7).
4. **LeetCode 78 / 46 — Subsets, Permutations** — https://leetcode.com/problems/subsets/ , https://leetcode.com/problems/permutations/

Core:

5. **CSES 1624 — Chessboard and Queens** — https://cses.fi/problemset/task/1624
6. **CSES 2165 — Tower of Hanoi** — https://cses.fi/problemset/task/2165
7. **Codeforces 550B — Preparing Olympiad** — https://codeforces.com/problemset/problem/550/B (choose a subset satisfying several constraints).
8. **LeetCode 39 — Combination Sum** — https://leetcode.com/problems/combination-sum/
9. **LeetCode 51 — N-Queens** — https://leetcode.com/problems/n-queens/ (print boards, general n).

Challenge:

10. **CSES 1625 — Grid Path Description** — https://cses.fi/problemset/task/1625 (pruning is everything here).
11. **CSES 1628 — Meet in the Middle** — https://cses.fi/problemset/task/1628 (n = 40 subset sums).

10. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 5 "Complete search" (generating subsets and permutations, backtracking, pruning, meet in the middle). Free PDF: https://cses.fi/book/book.pdf
- Steven & Felix Halim, *Competitive Programming 4*, Book 1, Section 3.2 "Complete Search" (recursive backtracking, pruning tips).
- USACO Guide, Bronze "Complete Search with Recursion": https://usaco.guide/bronze/complete-rec
- CP-Algorithms, "Generating all K-combinations": https://cp-algorithms.com/combinatorics/generating_combinations.html
- Next lecture (Week 9) revisits subset enumeration using bitmasks; Week 14 adds memoization to recursion; Week 26 turns exponential subset searches into bitmask DP.